5.1 **JavaScript Functions:**

5.1.1 Defining function (with and without parameters)

A JavaScript function is a block of code designed to perform a particular task.

A JavaScript function is executed when "something" invokes it (calls it).

```
function myFunction(p1, p2) {
  return p1 * p2;   // The function returns the product of p1 and p2
}
```

A JavaScript function is defined with the function keyword, followed by a **name**, followed by parentheses **()**.

Function names can contain letters, digits, underscores, and dollar signs (same rules as variables).

The parentheses may include parameter names separated by commas:
(*parameter1, parameter2, ...*)

The code to be executed, by the function, is placed inside curly brackets: **{}**

```
function name(parameter1, parameter2, parameter3) {
  // code to be executed
}
```

Function **parameters** are listed inside the parentheses () in the function definition.

Function **arguments** are the **values** received by the function when it is invoked.

Inside the function, the arguments (the parameters) behave as local variables.

5.1.2 Function Call

The code inside the function will execute when "something" **invokes** (calls) the function:

- When an event occurs (when a user clicks a button)
- When it is invoked (called) from JavaScript code
- Automatically (self invoked)

5.1.3 Function Return

When JavaScript reaches a return statement, the function will stop executing.

If the function was invoked from a statement, JavaScript will "return" to execute the code after the invoking statement.

Functions often compute a **return value**. The return value is "returned" back to the "caller":

```
let x = myFunction(4, 3);   // Function is called, return value will end up in x

function myFunction(a, b) {
  return a * b;           // Function returns the product of a and b
}
```

The result in x will be:

12

- **Why Functions?**

You can reuse code: Define the code once, and use it many times.

You can use the same code many times with different arguments, to produce different results.

5.1.4 Redirect a Webpage

There are a couple of ways to redirect to another webpage with JavaScript. The most popular ones are location.href and location.replace:

```
// Simulate a mouse click:
window.location.href = "http://www.w3schools.com";

// Simulate an HTTP redirect:
window.location.replace("http://www.w3schools.com");
```

The difference between href and replace, is that replace() removes the URL of the current document from the document history, meaning that it is not possible to use the "back" button to navigate back to the original document.

```
<html>

<body>

<h2>Redirect to a Webpage</h2>

<p>The replace() method replaces the current document with a new one:</p>

<button onclick="myFunction()">Replace document</button>

<script>

function myFunction() {

  location.replace("https://www.w3schools.com")

}

</script>

</body>

</html>
```

5.2 Dialog Box

5.2.1 Alert Box

An alert box is often used if you want to make sure information comes through to the user.
When an alert box pops up, the user will have to click "OK" to proceed.

Syntax

```
window.alert("sometext");
```

The window.alert() method can be written without the window prefix.

Example:

```
<html>
<body>

<h2>JavaScript Alert</h2>

<button onclick="myFunction()">Try it</button>

<script>
function myFunction() {
  alert("I am an alert box!");
}
</script>

</body>
</html>
```

5.2.2 Confirm Box

A confirm box is often used if you want the user to verify or accept something.

When a confirm box pops up, the user will have to click either "OK" or "Cancel" to proceed.

If the user clicks "OK", the box returns true. If the user clicks "Cancel", the box returns false.

Syntax

```
window.confirm("sometext");
```

The window.confirm() method can be written without the window prefix.

Example:

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript Confirm Box</h2>
<button onclick="myFunction()">Try it</button>
<p id="demo"></p>
<script>
function myFunction() {
var txt;
if (confirm("Press a button!")) {
txt = "You pressed OK!";
} else {
```

```
txt = "You pressed Cancel!";
}
document.getElementById("demo").innerHTML = txt;
}
</script>
</body>
</html>
```

5.2.3 Prompt Box

A prompt box is often used if you want the user to input a value before entering a page.

When a prompt box pops up, the user will have to click either "OK" or "Cancel" to proceed after entering an input value.

If the user clicks "OK" the box returns the input value. If the user clicks "Cancel" the box returns null.

Syntax

```
window.prompt("sometext","defaultText");
```

The window.prompt() method can be written without the window prefix.

Example:

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript Prompt</h2>
<button onclick="myFunction()">Try it</button>
<p id="demo"></p>
<script>
function myFunction() {
let text;
let person = prompt("Please enter your name:", "Harry Potter");
if (person == null || person == "") {
text = "User cancelled the prompt.";
} else {
text = "Hello " + person + "! How are you today?";
}
document.getElementById("demo").innerHTML = text;
}
</script>
</body>
</html>
```

5.3 JavaScript Form Validation

5.3.1 Basic Data Validation

```
 <html>
<body>
<script language="javascript">
function form_validation()
{
  name=document.form1.uname.value;
  password=document.form1.pass_word.value;
  if(name=="")
  {
        alert("name is required");
              return false;
  }
  if(password=="")
  {
        alert("please enter password");
              return false;
}
}
</script>

        <form name="form1" method="post" onsubmit= "form_validation();">
        student name:<input type="text" name="uname">
        <br>
        password:<input type="password" name="pass_word">
        <br>
        <input type="submit" value="submit">
        </form>
        </body>
</html>
```

5.3.2 Data Format Validation

• Sometimes it is necessary to check data for correct format and value. For example, an email address must enter in the proper format, the value must be a number, etc. This type of validation needs appropriate logic to test correctness of data.

a) Number Validation

• JavaScript can be used to validate input for numeric value only.

- The isNaN() (Not-A-Number) function is used for this purpose.

• The isNaN() function determines whether a given value is an illegal number or not.

- It returns true if a value is a not number, otherwise returns false.

Following example check whether inputted value is number or not.

EXAMPLE:

```
<html>
<body>
<script>
function num_validation()
{
no = document.getElementById("no").value;
if (isNaN(no))
{
alert( "Input is not number");
}
else
{
alert( "Input is number");
}
}
</script>
<input type="number" id="no" >
<Button type="button" onclick="num_validation()">Submit</button>
</body>
</html>
```

b) Email validation:

• JavaScript can validate the email format.

• Email address follows various criteria such as:

- Must contain the @ and . character
- Minimum one character before and after the @
- Minimum two characters after . (dot).

Example:

```
<html>
<body>
<script type = "text/javascript">
function email_validation()
{
//alert("Please enter correct email ID");
Email_id = document.getElementById("email").value;
at_position = Email_id.indexOf("@");
dot_position = Email_id.lastIndexOf(".");
if((at_position<1)||((dot_position-at_position)<2))
{
alert("Please enter correct email ID");
return false;
}
else
{
alert("correct");
return true ;
}
}
</script>
<input id="email">
<button type="button" onclick="email_validation()">Submit</button>
</body>
</html>
```

c) Mobile no. validation:

- JavaScript validate mobile no. Following code validate a phone number of 10 digits.

```
<html>
<body>
<script>
function mobile_number_validation()
{
        var mobile_no_format=/^\d{10}$/;
        inputtxt=document.getElementById("mobile_no").value;
```

```
if(inputtxt.match(mobile_no_format))
{
   alert("success");
        return true;
}
else
{
        alert("please enter 10 digit only");
        return false;
}
}
</script>
Mobile no:<input id="mobile_no">
<button type="button" onclick="mobile_number_validation()">submit
</button>
</body>
</html>
```

d) Validates that a string has not all blank /whitespace characters:

```
<html>
<body>
<script>
function string_empty_validation()
{
x = document.getElementById("str").value;
x=x.trim();
if (x.length == 0)
{
alert("string has blank spaces only");
return false;
}
else
alert("success");
return true;
}
</script>
input string: <input id="str">
<button type="button" onclick="string_empty_validation()">Submit</button>
</body>
</html?
```

(e) Validating string for minimum length:

• Following script ensure that user must enter string of minimum 4 characters.

```
<html>
<body>
<script>
  function string_length_validation()
  {
        string=document.getElementById("str").value;
        if(string.length < 4)
        {
         alert("minimum length ")
              return true;
        }
else
{
              alert("minimum length of sting is 4")
              return false;
        }
  }
</script>
input string:<input id="str">
<button type="button" onclick="string_length_validation()">Submit</button>
</body>
</html>
```

f) Checking string for all letters

• In some situations, input value must contain alphabets (A-Z or a-2) only. Following Expression.

```
<html>

<body>

<script>

function all_letter_validation()

{

letters=/^[A-Za-z]+$/;

string =document.getElementById("str").value;

if(string.match(letters))
```

```
{
alert("correct");
return true;
}
else
{
alert("enter only A-Z or a-z characters");
return false;
}
}
</script>
input string:<input id="str" >
<button type="button" onclick="all_letter_validation()">Submit</button>
</body>
</html>
```

(g) Validating name:

• The following code validates that user must enter first and last name. This is done using Regular Expression.

```
<html>
<body>
<script>
function name_validation()
{
var regName = /^[a-zA-Z]+[a-zA-Z]+$/;
string=document.getElementById("str").value;
```

```
if(string.match(regName))
{
alert('valid name given');
}
else {
alert('In Valid name given');
}
}
</script>
input name:<input id="str">
<button type="button" onclick="name_validation()"> Submit </button>
</body>
</html>
```